Python Programming Assignment Manual

Part 2: Lists, Dictionaries, Tuples & Sets

5 problems per data structure, ranging from foundational usage to applied, real-world challenges.

SECTION 8 — LISTS

1. Create and Print a List [Easy]

Problem Statement: Write a program to create a list of N integers (taken as input) and print the list along with its length.

Objective: Understand list creation and basic properties (length) of a list.

Input Format: An integer N, followed by N integers.

Output Format: The list itself, followed by its length on a new line.

Constraints: 0 <= N <= 1000

Example Input

```
3
10 20 30
```

Example Output

```
[10, 20, 30]
Length: 3
```

Explanation The 3 integers are collected into a list and printed as-is, then len() gives the count of elements.

Edge Cases to Consider: N=0 (empty list, length 0).

Additional Test Cases:

```
1) N=1, [5] -> [5]
   Length: 1
```

Hint: *Use a loop with append(), or build the list directly with list comprehension / input().split().*

Intuition & Analogy

> ***Analogy:*** *A list is like a train with numbered coaches. Creating the list is coupling coaches together one by one; the length is simply counting how many coaches are attached.*

2. Update Even-Indexed Elements [Medium]

Problem Statement: Given a list of integers, write a program to double the value of every element located at an even index (0, 2, 4, ...) and leave odd-indexed elements unchanged.

Objective: Practice indexed access, modification in place, and iterating with index tracking.

Input Format: A list of N integers.

Output Format: The modified list.

Constraints: 1 <= N <= 10^4

Example Input

```
[1, 2, 3, 4, 5]
```

Example Output

```
[2, 2, 6, 4, 10]
```

Explanation Indices 0, 2, 4 hold values 1, 3, 5 which get doubled to 2, 6, 10. Indices 1 and 3 (values 2, 4) stay unchanged.

Edge Cases to Consider: List of length 1 (only index 0, gets doubled).

Additional Test Cases:

```
1) [7] -> [14]
2) [1, 1] -> [2, 1]
```

Hint: *Use range(len(list)) so you have access to the index, and check index % 2 == 0.*

Intuition & Analogy

> ***Analogy:*** *Think of a row of lockers numbered from 0. You only open every other locker — the even-numbered ones — and double whatever's inside, leaving the odd lockers untouched.*

3. Real-World: Shopping Cart Manager [Medium]

Problem Statement: Simulate a shopping cart using a list. Write a function `manage_cart(cart, action, item)` where action is either 'add' or 'remove'. Adding appends the item; removing deletes the first occurrence of the item if present (do nothing if it isn't).

Objective: Apply list methods (append, remove) to a practical scenario, including safe handling of missing items.

Input Format: A list `cart`, a string `action`, and a string `item`.

Output Format: The updated cart list.

Constraints: 0 <= len(cart) <= 100

Example Input

```
cart=['Bread', 'Milk'], action='add', item='Eggs'
```

Example Output

```
['Bread', 'Milk', 'Eggs']
```

Explanation: Since the action is 'add', 'Eggs' is appended to the end of the existing cart.

Edge Cases to Consider: Removing an item that does not exist in the cart (should not raise an error).

Additional Test Cases:

```
1) cart=['Bread'], action='remove', item='Milk' -> ['Bread']
```

Hint: *Use `if item in cart:` before calling remove() to avoid a ValueError.*

Intuition & Analogy

> ***Analogy:*** *Think of an actual shopping cart at a store. Adding an item means placing it in the cart. Removing means checking if it's actually in the cart before you take it out — you can't remove what was never placed there.*

4. Reverse a List Using Slicing [Hard]

Problem Statement: Write a program to reverse a given list without using the built-in reverse() method or the reversed() function — use slicing only.

Objective: Master Python's slice notation, specifically the step parameter, to manipulate list order.

Input Format: A list of N integers.

Output Format: The reversed list.

Constraints: 0 <= N <= 10^4

Example Input

```
[1, 2, 3, 4, 5]
```

Example Output

```
[5, 4, 3, 2, 1]
```

Explanation The slice [::-1] starts from the end and steps backward through the whole list, effectively reversing it.

Edge Cases to Consider: Empty list (reversed is still empty), single-element list (unchanged).

Additional Test Cases:

```
1) [] -> []
2) [9] -> [9]
```

Hint: *Recall that list[start:stop:step] — a step of -1 traverses the list backward.*

Intuition & Analogy

> ***Analogy:*** *Picture rewinding a cassette tape. The slice [::-1] is like pressing rewind and playing the tape from the very end back to the beginning, without ever touching the physical order of the tape itself.*

5. Flatten a Nested List [Hard]

Problem Statement: Write a function `flatten(nested_list)` that takes a list which may contain other lists nested to any depth, and returns a single flat list containing all the individual (non-list) elements in order.

Objective: Understand nested lists and apply recursion (or an explicit stack) to traverse arbitrary depth.

Input Format: A nested list of integers.

Output Format: A single flat list of integers.

Constraints: Nesting depth <= 20, total elements <= 10^4

Example Input

```
[1, [2, 3, [4, 5]], 6]
```

Example Output

```
[1, 2, 3, 4, 5, 6]
```

Explanation The outer list has elements 1, a nested list, and 6. The nested list itself contains 2, 3, and another nested list [4, 5]. Flattening unpacks every level into a single sequence.

Edge Cases to Consider: A list that is already flat (no nesting), an empty nested list inside (e.g. [1, [], 2]).

Additional Test Cases:

```
1) [1, [], 2] -> [1, 2]
2) [[[1]]] -> [1]
```

Hint: *For each element, check isinstance(element, list). If true, recursively flatten it and extend the result; otherwise append it directly.*

Intuition & Analogy

> ***Analogy:*** *Think of a set of Russian nesting dolls (matryoshka). Flattening means opening every doll you find, no matter how many layers deep, and lining up every individual doll on the table in the order you discovered them.*

SECTION 9 — DICTIONARIES

1. Create a Dictionary and Access Values [Easy]

Problem Statement: Write a program to create a dictionary mapping fruit names to their prices, then print the price of a specific fruit given as input.

Objective: Understand dictionary creation (key-value pairs) and basic key-based access.

Input Format: A dictionary literal of fruit:price pairs, and a fruit name to look up.

Output Format: The price of the given fruit.

Constraints: 1 <= number of fruits <= 100

Example Input

```
{'apple': 50, 'banana': 20}, lookup='banana'
```

Example Output

```
20
```

Explanation: The key 'banana' maps directly to the value 20 in the dictionary.

Edge Cases to Consider: Looking up a fruit that doesn't exist in the dictionary.

Additional Test Cases:

```
1) {'apple': 50}, lookup='apple' -> 50
```

Hint: *Use dictionary[key] for direct access, or .get(key) to safely handle missing keys.*

Intuition & Analogy

> ***Analogy:*** *A dictionary is like a real language dictionary. The key is the word you look up, and the value is its definition sitting right next to it — you don't have to read the whole book, you jump straight to the word.*

2. Update and Add Key-Value Pairs [Medium]

Problem Statement: Write a function `update_inventory(inventory, item, quantity)` that adds `quantity` to the existing quantity of `item` if it's already a key in the dictionary, or creates the key with that quantity if it isn't.

Objective: Practice modifying dictionaries and safely handling both existing and new keys using .get() or 'in'.

Input Format: A dictionary `inventory`, a string `item`, and an integer `quantity`.

Output Format: The updated inventory dictionary.

Constraints: quantity can be positive or negative, resulting quantity >= 0

Example Input

```
inventory={'pen': 5}, item='pen', quantity=3
```

Example Output

```
{'pen': 8}
```

Explanation 'pen' already exists with quantity 5, so the new quantity 3 is added to it, giving 8.

Edge Cases to Consider: Adding an item that doesn't exist yet in the inventory.

Additional Test Cases:

```
1) inventory={'pen': 5}, item='pencil', quantity=4 -> {'pen': 5, 'pencil': 4}
```

Hint: *Use inventory[item] = inventory.get(item, 0) + quantity to handle both cases in one line.*

Intuition & Analogy

> ***Analogy:*** *Think of a warehouse stock ledger. If the item already has a shelf, you just add more units to the existing count. If it's a brand-new item, you create a new shelf entry and log the units there.*

3. Real-World: Word Frequency Counter [Medium]

Problem Statement: Write a function `word_frequency(sentence)` that takes a sentence and returns a dictionary mapping each word (lowercased) to the number of times it appears.

Objective: Combine string splitting with dictionary iteration to build a frequency map.

Input Format: A string sentence.

Output Format: A dictionary of word:count pairs.

Constraints: 1 <= len(sentence) <= 10^4

Example Input

```
'the cat sat on the mat the cat ran'
```

Example Output

```
{'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1, 'ran': 1}
```

Explanation 'the' appears 3 times and 'cat' appears 2 times; every other word appears once. Splitting on whitespace gives the individual words to count.

Edge Cases to Consider: Sentence with mixed casing (e.g. 'The THE' should all count as one word).

Additional Test Cases:

```
1) 'hi Hi HI' -> {'hi': 3}
```

Hint: *Use sentence.lower().split() to normalize the words first, then count using a dictionary or collections.Counter.*

Intuition & Analogy

> ***Analogy:*** *Imagine a librarian tallying how many times each book title is checked out. Every time a title comes up, whether it's on the shelf already or a first-time entry, a tally mark gets added next to its name.*

4. Merge Two Dictionaries with Conflict Resolution [Hard]

Problem Statement: Write a function `merge_dicts(d1, d2)` that merges two dictionaries. If a key exists in both, the merged value should be the SUM of the two values (assume numeric values).

Objective: Practice dictionary iteration, key-existence checks, and combining data from multiple sources.

Input Format: Two dictionaries d1 and d2 with numeric values.

Output Format: A single merged dictionary.

Constraints: 1 <= len(d1), len(d2) <= 10^3

Example Input

```
d1={'a': 10, 'b': 5}, d2={'b': 3, 'c': 7}
```

Example Output

```
{'a': 10, 'b': 8, 'c': 7}
```

Explanation 'a' only exists in d1 so it stays 10. 'b' exists in both (5 and 3), so it's summed to 8. 'c' only exists in d2 so it stays 7.

Edge Cases to Consider: One of the dictionaries is empty; no overlapping keys at all.

Additional Test Cases:

```
1) d1={}, d2={'x': 1} -> {'x': 1}
```

Hint: *Start with a copy of d1, then loop through d2's items, adding to existing keys or inserting new ones.*

Intuition & Analogy

> ***Analogy:*** *Think of merging two team's expense reports for the same month. If both teams logged an expense under the same category, the amounts get combined into one total; categories unique to one team just carry over as-is.*

5. Real-World: Nested Student Records [Hard]

Problem Statement: Given a dictionary mapping student names to a nested dictionary of their subject:marks pairs, write a function `average_marks(records)` that returns a dictionary mapping each student's name to their average marks across all subjects, rounded to 2 decimal places.

Objective: Navigate nested dictionaries, iterate over inner structures, and aggregate values.

Input Format: A nested dictionary: {student_name: {subject: marks, ...}, ...}

Output Format: A dictionary mapping student_name to their average marks.

Constraints: 1 <= number of students <= 100, 1 <= number of subjects per student <= 20

Example Input

```
{'Alice': {'Math': 90, 'Science': 80}, 'Bob': {'Math': 70, 'Science': 60, 'Art': 100}}
```

Example Output

```
{'Alice': 85.0, 'Bob': 76.67}
```

Explanation: Alice's average is (90+80)/2 = 85.0. Bob's average is (70+60+100)/3 = 76.666... which rounds to 76.67.

Edge Cases to Consider: A student with only one subject (average equals that single mark).

Additional Test Cases:

```
1) {'Cara': {'Math': 100}} -> {'Cara': 100.0}
```

Hint: *For each student, use sum(subject_dict.values()) / len(subject_dict) to compute the average.*

Intuition & Analogy

> ***Analogy:*** *Picture a school's report card system stored in labeled folders. Each student has their own folder, and inside each folder are individual subject sheets. To find a student's average, you open their folder, add up every subject sheet's score, and divide by how many sheets are inside.*

SECTION 10 — TUPLES

1. Create a Tuple and Access Elements [Easy]

Problem Statement: Write a program to create a tuple of N integers and print the element at a given index.

Objective: Understand tuple creation and that tuples support the same indexed access as lists, but are immutable.

Input Format: N integers to form the tuple, and an index to access.

Output Format: The element at the given index.

Constraints: 1 <= N <= 1000, 0 <= index < N

Example Input

```
(10, 20, 30, 40), index=2
```

Example Output

```
30
```

Explanation Index 2 in the tuple (0-indexed) holds the value 30.

Edge Cases to Consider: Accessing index 0 (first element) or the last valid index N-1.

Additional Test Cases:

```
1) (5,), index=0 -> 5
```

Hint: *Tuple indexing works exactly like list indexing: tuple[index].*

Intuition & Analogy

> ***Analogy:*** *A tuple is like a sealed jar of labeled slots — you can look inside and read what's in slot number 2, but you can never swap out what's sitting there.*

2. Tuple Packing and Unpacking [Medium]

Problem Statement: Write a function `swap_and_pack(a, b, c)` that packs the three arguments into a tuple, then unpacks them into three new variables in reversed order, and returns the reversed tuple.

Objective: Practice tuple packing (combining values into a tuple) and unpacking (extracting values into separate variables).

Input Format: Three values a, b, c.

Output Format: A tuple with the values in reversed order.

Constraints: None specific — any hashable values.

Example Input

```
a=1, b=2, c=3
```

Example Output

```
(3, 2, 1)
```

Explanation The values are packed as (1, 2, 3), then unpacked in reverse into new variables holding 3, 2, and 1, which are packed again into the returned tuple.

Edge Cases to Consider: All three values being identical (e.g. 5, 5, 5).

Additional Test Cases:

```
1) a='x', b='y', c='z' -> ('z', 'y', 'x')
```

Hint: *Packing: t = (a, b, c). Unpacking: x, y, z = t. Then return (z, y, x).*

Intuition & Analogy

> ***Analogy:*** *Think of packing three items into a single gift box (packing), then having a friend open the box and hand each item to a different person in reverse order (unpacking) — a compact way to move multiple values around together.*

3. Slicing a Tuple [Medium]

Problem Statement: Write a program that, given a tuple of integers, returns a new tuple containing only the middle elements — excluding the first and last element.

Objective: Apply slice notation to tuples, reinforcing that slicing a tuple returns a new tuple (not a list).

Input Format: A tuple of N integers.

Output Format: A tuple with the first and last elements removed.

Constraints: 0 <= N <= 10^4

Example Input

```
(1, 2, 3, 4, 5)
```

Example Output

```
(2, 3, 4)
```

Explanation Slicing [1:-1] skips index 0 and the final index, keeping everything strictly in between.

Edge Cases to Consider: Tuple of length 0 or 1 (result should be an empty tuple), tuple of length 2 (result is empty since nothing is 'in between').

Additional Test Cases:

```
1) () -> ()
2) (1, 2) -> ()
```

Hint: *Use tuple[1:-1] — the same slicing rules that apply to lists and strings apply to tuples.*

Intuition & Analogy

> ***Analogy:*** *Imagine a sandwich: the first and last elements are the two bread slices, and slicing [1:-1] pulls out just the filling in between, leaving the crusts behind.*

4. Real-World: GPS Coordinate Validator [Hard]

Problem Statement: Given a tuple of GPS coordinates, where each coordinate is itself a tuple of (latitude, longitude), write a function `validate_coordinates(coords)` that returns a tuple containing only the valid coordinates — valid meaning -90 <= latitude <= 90 and -180 <= longitude <= 180.

Objective: Work with nested (immutable) tuples, iterate through them, and apply validation logic without modifying the original data.

Input Format: A tuple of (latitude, longitude) tuples.

Output Format: A tuple containing only the valid coordinate tuples, in original order.

Constraints: 1 <= number of coordinates <= 10^3

Example Input

```
((45.0, 90.0), (200.0, 10.0), (-90.0, -180.0))
```

Example Output

```
((45.0, 90.0), (-90.0, -180.0))
```

Explanation The second coordinate has latitude 200.0, which exceeds the valid range of -90 to 90, so it's excluded. The other two fall within valid bounds.

Edge Cases to Consider: A coordinate exactly on the boundary (e.g. latitude=90.0, longitude=180.0) should be treated as valid.

Additional Test Cases:

```
1) ((91.0, 0.0),) -> ()
```

Hint: *Iterate through the outer tuple, unpack each inner tuple into lat, lon, and check both conditions before including it.*

Intuition & Analogy

> ***Analogy:*** *Think of a ship's navigator double-checking a list of charted coordinates before setting sail. Any coordinate that falls off the edge of the map (outside valid lat/lon bounds) gets flagged and left out of the final approved route — the original chart itself is never altered.*

5. Count Occurrences in a Tuple of Tuples [Hard]

Problem Statement: Given a tuple where each element is itself a tuple of strings (representing tags on different posts), write a function `tag_counts(posts)` that returns a dictionary mapping each unique tag to how many posts it appears in.

Objective: Combine nested tuple traversal with dictionary aggregation.

Input Format: A tuple of tuples of strings.

Output Format: A dictionary mapping tag to count.

Constraints: 1 <= number of posts <= 10^3, 1 <= tags per post <= 20

Example Input

```
(('python', 'ai'), ('python', 'web'), ('ai',))
```

Example Output

```
{'python': 2, 'ai': 2, 'web': 1}
```

Explanation 'python' appears in the 1st and 2nd post tuples (count 2). 'ai' appears in the 1st and 3rd (count 2). 'web' appears only in the 2nd (count 1). Note: even though a tag appears once within a single post, we count posts it appears in, not raw occurrences.

Edge Cases to Consider: A post with no tags (an empty inner tuple).

Additional Test Cases:

```
1) (('x',), ()) -> {'x': 1}
```

Hint: *Loop through the outer tuple, then loop through each inner tuple's tags, updating a dictionary count for each tag encountered.*

Intuition & Analogy

> ***Analogy:*** *Imagine a set of sealed photo albums, each containing several hashtag stickers on its cover. To find how popular each hashtag is, you go album by album, note down every unique sticker you see, and tally which albums it showed up on.*

SECTION 11 — SETS

1. Create a Set and Add Elements [Easy]

Problem Statement: Write a program to create an empty set, add N given integers to it one at a time, and print the final set.

Objective: Understand set creation and the add() method, and observe that duplicates are automatically ignored.

Input Format: N integers to add to the set.

Output Format: The resulting set.

Constraints: 0 <= N <= 1000

Example Input

```
5 3 5 1 3
```

Example Output

```
{1, 3, 5}
```

Explanation: Even though 5 and 3 are entered twice, a set only keeps unique values, so the final set has just {1, 3, 5}.

Edge Cases to Consider: Adding the same value repeatedly (set size never grows beyond 1 for that value).

Additional Test Cases:

```
1) 7 7 7 -> {7}
```

Hint: *Start with s = set(), then call s.add(value) for each input value.*

Intuition & Analogy

> ***Analogy:*** *A set is like a guest list at an exclusive party — no matter how many times someone tries to RSVP, their name only ever appears once on the final list.*

2. Remove Duplicates Using a Set [Medium]

Problem Statement: Write a program to remove duplicate values from a list using a set, then return the result as a sorted list.

Objective: Apply sets as a practical tool for deduplication, and convert back to a list with a defined order.

Input Format: A list of integers.

Output Format: A sorted list with duplicates removed.

Constraints: 0 <= N <= 10^4

Example Input

```
[4, 2, 4, 1, 2, 3]
```

Example Output

```
[1, 2, 3, 4]
```

Explanation Converting the list to a set collapses the duplicate 4s and 2s down to single entries, and sorting arranges them in ascending order.

Edge Cases to Consider: An empty list, or a list where every element is already unique.

Additional Test Cases:

```
1) [] -> []
2) [9, 8, 7] -> [7, 8, 9]
```

Hint: *Use sorted(set(the_list)) — converting to a set removes duplicates, and sorted() returns an ordered list.*

Intuition & Analogy

> ***Analogy:*** *Think of pouring a jar of mixed, repeated marbles into a strainer that only lets one marble of each color through, then lining up the surviving marbles by color.*

3. Set Operations — Union, Intersection, Difference [Medium]

Problem Statement: Write a function `set_operations(a, b)` that takes two sets and returns a tuple of three sets: their union, their intersection, and their difference (a - b), in that order.

Objective: Practice the core mathematical set operations available as built-in set methods/operators.

Input Format: Two sets a and b.

Output Format: A tuple: (union, intersection, difference).

Constraints: 0 <= len(a), len(b) <= 10^3

Example Input

```
a={1, 2, 3}, b={2, 3, 4}
```

Example Output

```
({1, 2, 3, 4}, {2, 3}, {1})
```

Explanation Union combines all unique elements from both sets. Intersection keeps only elements present in both. Difference (a - b) keeps elements in a that are NOT in b, which is just {1}.

Edge Cases to Consider: Two completely disjoint sets (intersection is empty).

Additional Test Cases:

```
1) a={1}, b={2} -> ({1, 2}, set(), {1})
```

Hint: *Use a | b for union, a & b for intersection, and a - b for difference.*

Intuition & Analogy

> ***Analogy:*** *Think of two overlapping Venn diagram circles. Union is everything covered by either circle. Intersection is just the overlapping middle region. Difference (a - b) is the crescent-shaped part of circle a that circle b doesn't cover.*

4. Real-World: Common Friends Finder [Hard]

Problem Statement: Given a dictionary mapping each person's name to a set of their friends' names, write a function `mutual_friends(network, person1, person2)` that returns the set of friends common to both person1 and person2.

Objective: Apply set intersection to a realistic social-network scenario, combined with dictionary lookups.

Input Format: A dictionary network of {name: set_of_friend_names}, and two names person1, person2.

Output Format: A set of mutual friends' names.

Constraints: 1 <= number of people <= 10^3

Example Input

```
network={'Alice': {'Bob', 'Cara'}, 'Dan': {'Bob', 'Eve'}}, person1='Alice', person2='Dan'
```

Example Output

```
{'Bob'}
```

Explanation Alice's friend set is {'Bob', 'Cara'} and Dan's is {'Bob', 'Eve'}. The only name appearing in both sets is 'Bob'.

Edge Cases to Consider: One of the two people has no friends in common with the other (returns an empty set); a person not present in the network at all.

Additional Test Cases:

```
1) network={'A': {'X'}, 'B': {'Y'}}, person1='A', person2='B' -> set()
```

Hint: *Look up each person's friend set from the dictionary using .get(name, set()) to safely handle a missing person, then apply the & operator.*

Intuition & Analogy

> ***Analogy:*** *Picture two people each holding their own phone contact list. To find mutual friends, you simply lay both lists side by side and circle every name that appears on both — that circled group is the intersection.*

5. Real-World: Deduplicate Event Log Tags Across Sessions [Hard]

Problem Statement: Given a list of sets, where each set represents the unique event tags logged during one user session, write a function `all_unique_tags(sessions)` that returns a single set containing every distinct tag seen across all sessions combined.

Objective: Apply set union across multiple sets and practice iterating over a collection of sets to aggregate data.

Input Format: A list of sets of strings.

Output Format: A single set containing all distinct tags.

Constraints: 0 <= number of sessions <= 10^3

Example Input

```
[{'login', 'click'}, {'click', 'purchase'}, {'logout'}]
```

Example Output

```
{'login', 'click', 'purchase', 'logout'}
```

Explanation 'click' appears in two different sessions but only counts once in the combined result, since sets automatically eliminate duplicates during union.

Edge Cases to Consider: An empty list of sessions (result is an empty set), a session with no tags (an empty set inside the list).

Additional Test Cases:

```
1) [] -> set()
2) [{'a'}, set()] -> {'a'}
```

Hint: *Start with an empty set and use the union() method (or |=) inside a loop to combine each session's tags into the result.*

Intuition & Analogy

> ***Analogy:*** *Think of collecting stickers from several different sticker packs across a week. At the end, you dump every sticker onto one table, but since duplicate stickers just overlap, what you're really left with is a display of every distinct design you ever collected.*